🔐 DevSecOps Security Tools

---

1. Static Application Security Testing (SAST)

These tools scan source code to detect vulnerabilities and coding flaws before deployment.

- **SonarQube** – Analyzes code for bugs, vulnerabilities, and code smells.
- **Semgrep** – Fast, syntax-aware code scanner with customizable rules.
- **Bandit** – Python-specific security linter.
- **ESLint + Security Plugins** – JavaScript/Node.js linting with security extensions.

---

2. Software Composition Analysis (SCA)

Analyzes third-party and open-source dependencies for known vulnerabilities.

- **Snyk** – Identifies and fixes vulnerabilities in dependencies, Docker images, and IaC.
- **OWASP Dependency-Check** – Scans for known vulnerable libraries using CVE data.
- **Trivy** – Scans code, containers, and IaC for vulnerabilities.
- **Retire.js** – JavaScript library vulnerability scanner.

PATHNEX
Shaping Careers, Building Futures

---

3. Container Security

Scans and secures container images and registries.

- **Clair** – Static analysis for Docker containers to find known vulnerabilities.
- **Anchore Engine** – Deep inspection of container images with policy enforcement.
- **Trivy** – Also excellent for container image scanning and fast CLI usage.
- **Docker Bench for Security** – Audits Docker host configurations against best practices.

---

4. Infrastructure-as-Code (IaC) Security

Detects misconfigurations in Terraform, CloudFormation, Kubernetes YAML, etc.

- **Checkov** – Scans Terraform, CloudFormation, Kubernetes for security misconfigurations.
- **Terrascan** – Helps detect compliance and security violations in IaC files.
- **KICS (Keep Infrastructure as Code Secure)** – Scans IaC for vulnerabilities.
- **TFLint** – Linter for Terraform focusing on code quality and security.

5. Dynamic Application Security Testing (DAST)

Simulates attacks on running applications to find vulnerabilities.

- **OWASP ZAP** – Automated scanner for web applications.
- **Burp Suite (Community Edition)** – Manual and semi-automated web vulnerability testing.
- **Nikto** – Web server scanner detecting outdated servers and insecure configurations.
- **Arachni** – High-performance Ruby-based web scanner.

---

6. Secrets Management & Detection

Protects sensitive data like API keys, passwords, and tokens.

- **HashiCorp Vault** – Centralized secrets management, encryption, and dynamic credentials.
- **CyberArk Conjur** – Manages secrets and machine identity securely for apps and containers.
- **Gitleaks** – Scans Git repositories for hardcoded secrets.
- **TruffleHog** – Detects sensitive data in git history.

PATHNEX
Shaping Careers, Building Futures

---

7. Runtime Security Monitoring

Monitors applications, containers, and infrastructure at runtime for suspicious behavior.

- **Falco** – Behavioral activity monitoring for containers and Kubernetes.
- **AppArmor / SELinux** – Mandatory access control systems for Linux.
- **Sysdig Secure** – Monitors container activity and detects anomalies.
- **Auditd** – Native Linux auditing tool to track system calls and events.

---

8. Security Logging & SIEM

Collects and analyzes security data from across the infrastructure.

- **ELK Stack (Elasticsearch, Logstash, Kibana)** – Centralized logging and visualization.
- **Fluentd + Elasticsearch** – Log forwarding and aggregation for container environments.
- **Security Onion** – Comprehensive network security monitoring platform.
- **Wazuh** – Security analytics, threat detection, file integrity monitoring.

9. Vulnerability Management

Centralizes and manages scanning, triage, and remediation workflows.

- **DefectDojo** – Aggregates findings from various tools and tracks remediation.
- **TheHive** – Incident response platform to manage and track security events.
- **Faraday** – Collaborative vulnerability analysis and management.

---

10. CI/CD Integration

Integrating DevSecOps tools into the pipeline for shift-left security.

- **Jenkins + Snyk/SonarQube Plugins** – For inline scanning during builds.
- **GitLab CI + Secret Scanning, SAST/DAST Stages** – Built-in DevSecOps support.
- **GitHub Actions + Trivy/Semgrep** – Automate scanning in pull requests.

✅ Suggested Learning Path

1. **Code Security (SAST)** – SonarQube, Semgrep
2. **Dependency Scanning (SCA)** – Snyk, Trivy
3. **Container Security** – Clair, Anchore, Docker Bench
4. **IaC Security** – Checkov, Terrascan
5. **DAST** – OWASP ZAP, Burp Suite
6. **Secrets Detection & Management** – Vault, Gitleaks, TruffleHog
7. **Runtime Protection** – Falco, AppArmor
8. **Logging & Monitoring** – ELK Stack, Wazuh
9. **Vuln Management** – DefectDojo
10. **CI/CD Integration** – GitLab/Jenkins with scanners

Shaping Careers, PATHNEX Building Futures

Practical use cases

Lab 1: Static Code Analysis (SAST) with SonarQube and Semgrep

Objective:

How to scan code for vulnerabilities using SonarQube and Semgrep.

Tools Required:

- SonarQube (Dockerized setup)
- Semgrep

Lab Steps:

1. **Set up SonarQube**:
    - Run SonarQube in Docker or locally.
    - Install the SonarQube plugin for your IDE (VS Code or IntelliJ).
    - Clone a vulnerable Java/Python repository from GitHub (e.g., "vulnerable code repo").
    - Analyze the code using SonarQube.
    - Review vulnerabilities, coding errors, and security issues highlighted.
2. **Semgrep Setup**:
    - Install Semgrep on a local machine or in CI/CD pipeline.
    - Scan the same repository with Semgrep.
    - Identify security issues such as hardcoded secrets or weak cryptography.
    - Discuss results and address how Semgrep custom rules can be created.
3. **Analysis & Reporting**:
    - Compare the findings of SonarQube vs. Semgrep.
    - Discuss strengths and weaknesses of each tool.

PATHNEX
Shaping Careers, Building Futures

Lab 2: Container Security with Trivy and Docker Bench

Objective:

How to scan Docker containers for vulnerabilities and best practices.

Tools Required:

- Docker
- Trivy
- Docker Bench for Security

Lab Steps:

1. **Trivy Container Scanning**:
   - Build or pull a Docker image (e.g., nginx or vulnerable container).
   - Scan the image using `trivy image <image-name>`.
   - Review the vulnerabilities reported and patch any issues by updating the base image or libraries.
2. **Docker Bench Setup**:
   - Run the `docker-bench-security` script on your Docker host to check for security misconfigurations.
   - Review the security findings, e.g., use of root user in containers, exposed ports, etc.
   - Apply security best practices and rerun the script to verify improvements.
3. **Discussion**:
   - Discuss which vulnerabilities are critical for containerized apps and how to mitigate them.
   - Mention how these tools can be integrated into CI/CD pipelines for ongoing checks.

Lab 3: Infrastructure as Code Security with Checkov and Terrascan

PATHNEX
Shaping Careers, Building Futures

Objective:

Scan Terraform and CloudFormation templates for security issues.

Tools Required:

- Terraform
- Checkov
- Terrascan

Lab Steps:

1. **Set Up a Simple Terraform Script**:
   - Write a basic Terraform configuration for an AWS EC2 instance.
   - Run `terraform init` and `terraform apply` to create the infrastructure.
2. **Checkov Scanning**:
   - Install Checkov and run it against the Terraform script: `checkov -d .`
   - Review the output to identify misconfigurations, such as open security groups, insecure IAM roles, etc.
   - Fix the vulnerabilities in the script and re-scan to verify.
3. **Terrascan Integration**:
   - Install Terrascan and run it on the Terraform script: `terrascan scan -i terraform -d .`
   - Compare the results from Checkov and Terrascan.

    - Discuss the differences in policy coverage and the tools' advantages.
4. **Continuous Integration**:
    - Integrate Checkov and Terrascan into Jenkins/GitLab pipeline to automate IaC scanning for every commit.

Lab 4: Dynamic Application Security Testing (DAST) with OWASP ZAP

Objective:

Perform security scanning on a live web application using OWASP ZAP.

Tools Required:

- OWASP ZAP
- A vulnerable web app (e.g., DVWA, OWASP Juice Shop)

Lab Steps:

1. **Setup OWASP ZAP**:
    - Download and install OWASP ZAP or run it as a Docker container.
    - Launch the ZAP proxy and configure your browser to route traffic through ZAP.
2. **Scan a Vulnerable Web Application**:
    - Set up a vulnerable web application like DVWA (Damn Vulnerable Web Application) or OWASP Juice Shop.
    - Perform a basic automated scan with ZAP.
    - Identify common vulnerabilities such as SQL Injection, XSS, or CSRF.
3. **Manual Scanning**:
    - Use the ZAP interface to perform manual security testing.
    - Explore tools like Spider, Active Scan, and Passive Scan.
    - Identify vulnerabilities manually, such as insecure headers or broken authentication.
4. **Reporting & Remediation**:
    - Generate a ZAP report detailing the findings.
    - Discuss common vulnerabilities found and remediation strategies.

Lab 5: Secrets Management with HashiCorp Vault and GitLeaks

Objective:

Securely manage and detect secrets in an application.

PATHNEX
Shaping Careers, Building Futures

Tools Required:

- HashiCorp Vault
- GitLeaks

Lab Steps:

1. **Setup HashiCorp Vault**:
   - Install HashiCorp Vault using Docker or directly on your system.
   - Start Vault in dev mode (`vault server -dev`).
   - Use Vault's CLI to create secrets (e.g., API keys, credentials).
   - Show how to securely store and access secrets using Vault.
2. **GitLeaks Setup**:
   - Install GitLeaks and scan a Git repository for exposed secrets.
   - If secrets are found, demonstrate how to remove them securely from Git history using `git filter-branch` or `BFG Repo-Cleaner`.
3. **Integrating Vault into Application**:
   - Modify a simple application (e.g., Node.js) to retrieve secrets from Vault during runtime instead of hardcoding them.
   - Discuss how this improves security by keeping secrets centralized and encrypted.
4. **CI/CD Integration**:
   - Show how to integrate Vault and GitLeaks in your CI pipeline to scan for secrets in code before pushing.

PATHNEX
Shaping Careers, Building Futures

Lab 6: Runtime Security with Falco

Objective:

Monitor and detect abnormal behavior in containers at runtime using Falco.

Tools Required:

- Kubernetes or Docker
- Falco

Lab Steps:

1. **Setup Falco on Docker or Kubernetes**:
   - Install Falco in a Docker container or Kubernetes pod.
   - Start monitoring system calls and container activities.
2. **Simulate Abnormal Behavior**:
   - Run a simple container and simulate malicious behavior, such as trying to escalate privileges or accessing sensitive files.
   - Observe how Falco detects and alerts on these activities.
3. **Alerting and Response**:
   - Configure Falco to send alerts via email, Slack, or a SIEM tool.

- Discuss how runtime security monitoring helps detect and respond to security incidents in real-time.

Lab 7: Vulnerability Management with TheHive

Objective:

Track, manage, and respond to vulnerabilities using TheHive platform.

Tools Required:

- TheHive
- Cortex (optional, for integrations)

Lab Steps:

1. **Set up TheHive**:
   - Install TheHive and access its web interface.
   - Create cases for different types of security incidents or vulnerabilities.
2. **Integrate with External Scanners**:
   - Integrate the tool with a vulnerability scanner like OWASP ZAP or Snyk.
   - Import findings into TheHive to create incidents.
3. **Incident Management**:
   - Track the status of vulnerabilities, assign them to team members, and add remediation notes.
   - Discuss best practices for handling vulnerability reports and incident response.

PATHNEX
Shaping Careers, Building Futures

Lab 8: Integrating Security into CI/CD (Jenkins/GitLab)

Objective:

Integrate the security tools into a CI/CD pipeline for automated security checks.

Tools Required:

- Jenkins or GitLab
- SonarQube, Checkov, Snyk, Trivy

Lab Steps:

1. **Set up Jenkins or GitLab CI**:
   - Create a simple pipeline that triggers on push to GitHub/GitLab.
   - Add stages for build, test, and deploy.
2. **Add Security Scanning**:

- Add SonarQube analysis to the build pipeline.
- Integrate Trivy to scan Docker images during the build process.
- Include Checkov or Snyk to scan IaC for vulnerabilities.

3. **Automatic Alerts**:
   - Configure Jenkins/GitLab to send alerts if any security issues are found during the pipeline.
   - Discuss how to integrate tools like Slack or email for notifications.

PATHNEX
Shaping Careers, Building Futures